

LECTURE 6

Process Concept and Process States

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The process concept: why a process is not the same thing as a program
- The process state diagram: New, Ready, Running, Waiting, Terminated — and what moves a process between them
- The Process Control Block (PCB): what the OS stores about every process, and why
- Process scheduling: scheduling queues and what a context switch actually costs
- Process creation with `fork()`, `exec()`, and `wait()`
- Process termination, including the zombie vs. orphan process distinction

The Process Concept

A process is a program in execution. A program is a static set of instructions sitting on disk; a process is a dynamic entity: instructions actively being fetched and executed, a program counter tracking what's next, a stack growing and shrinking as functions are called.

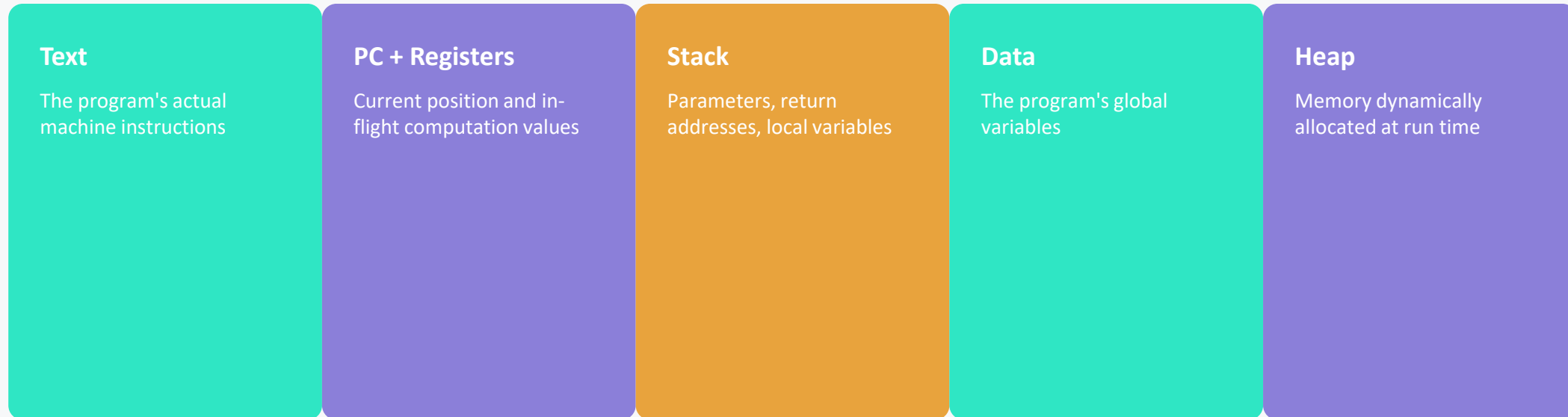
- Running the same executable twice (two terminal windows, both a shell) produces two separate processes
- Each has its own stack, its own heap, its own program counter, its own notion of “where am I right now”
- They share the same underlying program but are completely independent as far as the OS is concerned

A Recipe vs. the Act of Cooking

A program is like a recipe card. A fixed, passive set of instructions that doesn't change no matter how many times you read it. A process is like actually cooking from that recipe — a specific person, at a specific stove, at a specific step, with specific ingredients already measured out (the process's own memory and data). Two people can cook from the same recipe card at the same time, in two different kitchens, at two different steps — exactly how two processes can run the same program while being two entirely separate, independently-tracked processes.

What Makes Up a Process

A process, as the OS sees it, is made up of several distinct parts:

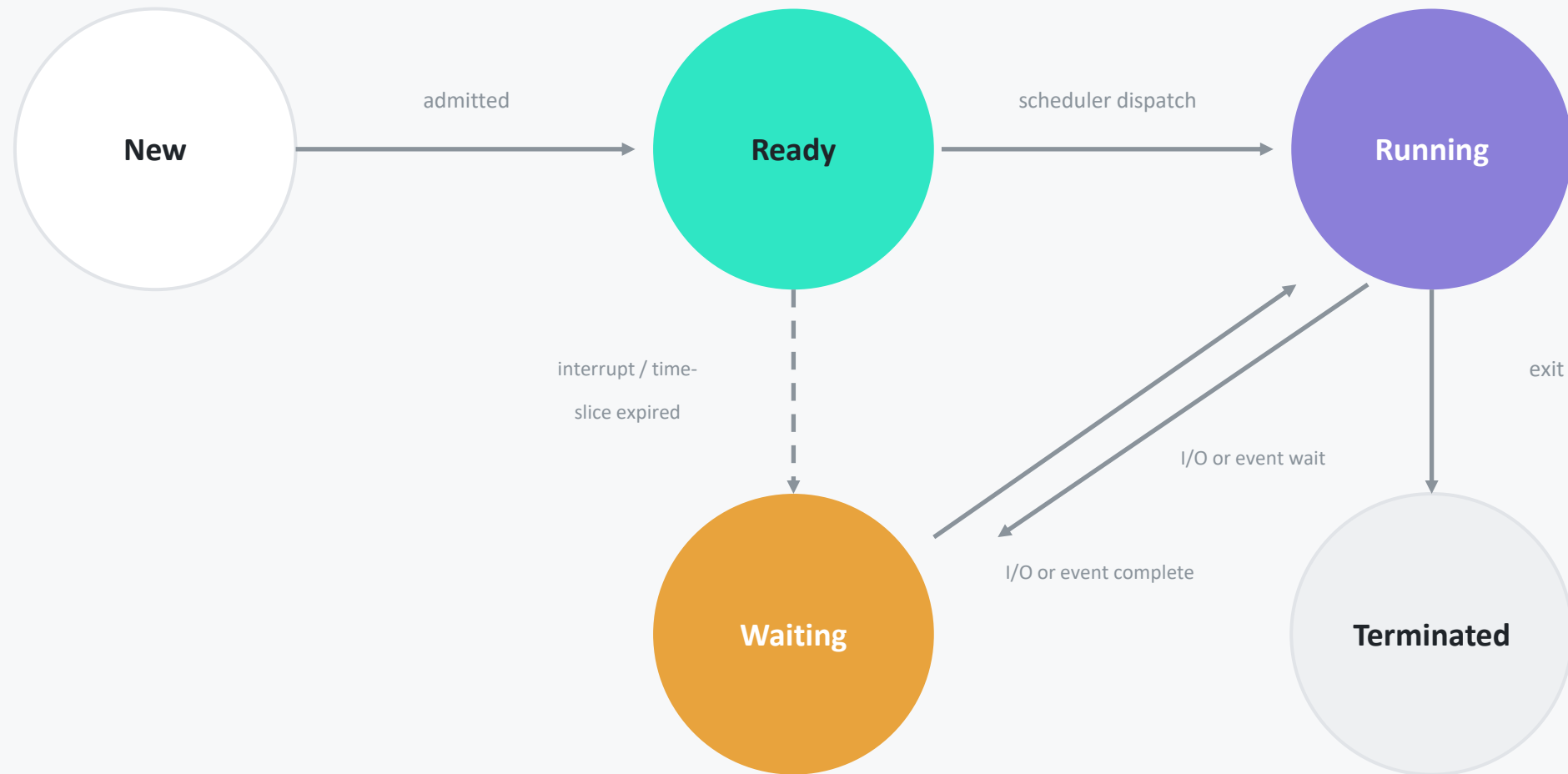


Process States

Over its lifetime a process moves through a well-defined set of states, and the OS is constantly tracking which state every process is currently in.

- New — the process is being created; the OS is setting up its bookkeeping but it hasn't been admitted to compete for the CPU
- Ready — the process has everything it needs to run and is waiting for the scheduler to pick it
- Running — the process's instructions are actually being executed right now (at most one per core)
- Waiting / Blocked — the process cannot proceed until some event occurs, most often I/O completing
- Terminated — the process has finished (or was forcibly ended) and is being removed from the system

The Process State Diagram



Walking Through Each Transition

- Admitted (New → Ready) — the OS finishes setup and adds the process to the ready queue
- Scheduler dispatch (Ready → Running) — the CPU scheduler picks this process and hands it the CPU
- Interrupt / time-slice expired (Running → Ready) — the process is forced off the CPU without finishing; it didn't choose to stop
- I/O or event wait (Running → Waiting) — the process requests something it can't have immediately and voluntarily gives up the CPU
- I/O or event complete (Waiting → Ready) — the blocking event has happened; it's eligible to run again but must still wait its turn
- Exit (Running → Terminated) — the process finishes its last instruction, and the OS reclaims its resources

A Waiting Process Can't Go Straight Back to Running

A common mix-up: drawing an arrow directly from Waiting back to Running. That's wrong — when an I/O operation completes, the process only becomes eligible to run again. It moves to Ready and must still be selected by the scheduler like every other ready process. On a busy system, a process can sit in Ready for a while even after the event it was waiting for has already happened.

The Process Control Block (PCB)

For every process it manages, the OS maintains a data structure called the Process Control Block (PCB) — the process's complete identity card, holding everything the OS needs to pause it, resume it later exactly where it left off, and make scheduling decisions about it.

- The PCB is why a process can be paused mid-instruction and resumed as if nothing happened
- Every piece of information needed to reconstruct its exact state sits in its PCB, maintained by the OS

PCB — Fields and Purpose

Field	What it stores	Why the OS needs it
Process state	New, Ready, Running, Waiting, or Terminated	Tells the OS what to do with this process next
Process ID (PID)	A unique identifier for this process	Lets the OS refer to this exact process unambiguously
Program counter	Address of the next instruction to execute	Lets execution resume at exactly the right place
CPU registers	Every general-purpose and status register	Must be restored exactly, or the resumed computation is wrong
Scheduling info	Priority, queue pointers, parameters	Lets the scheduler decide which Ready process runs next
Memory info	Base/limit registers, page or segment tables	Tells the OS exactly which memory belongs to this process
Accounting info	CPU time used, time limits, owner	Used for billing, limits, and performance monitoring
I/O status	Open files, allocated I/O devices	Lets the OS track what resources this process is using

Process Scheduling — Scheduling Queues

With potentially hundreds of processes and only a handful of cores, the OS organizes processes so the scheduler can efficiently find the right one to run next:

Job queue

Every process in the system, in every state, from creation until it terminates

Ready queue

Specifically the processes currently in the Ready state, waiting for a turn on the CPU

Device queues

One per I/O device, holding processes waiting for that device to finish or become available

A process moves between these queues constantly over its lifetime — the Ready queue when it can run, a device queue while waiting on I/O, and back to Ready once that I/O completes.

Context Switching

When the scheduler takes the CPU away from one process and gives it to another, the OS performs a context switch: it saves the current process's entire CPU state into its PCB, then loads the next process's previously-saved state from its PCB back into the CPU.

A Context Switch Is Pure Overhead

No useful work happens during a context switch. While the OS is busy saving one process's state and loading another's, neither process is making progress. Every microsecond spent on a context switch is a microsecond the CPU isn't running any application's actual code — which is exactly why operating systems work hard to make context switches fast, and why switching too frequently can hurt throughput even though each switch looks “fair.”

Process Creation

A running process can create new processes — the creating process is the parent, the new one is its child, forming a tree of processes going back to whatever process the OS started first at boot. Two Unix-associated operations do the actual work, plus a third to wait for a child:

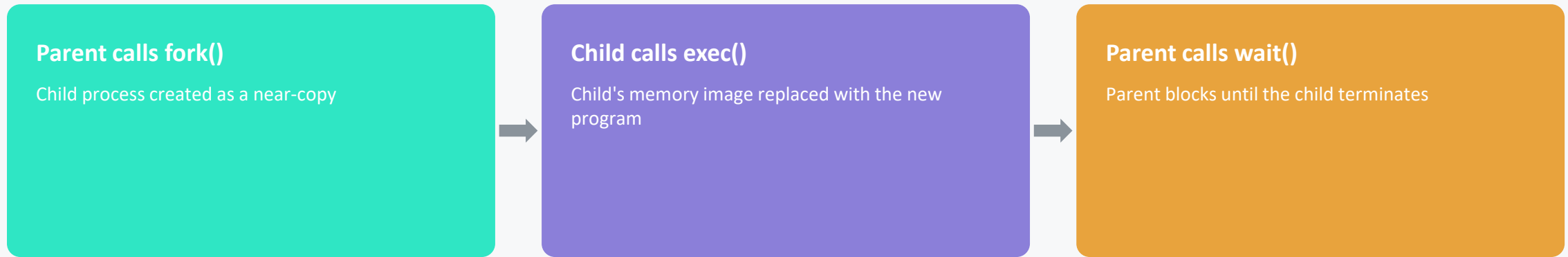
- `fork()` — creates a new process that is (initially) a near-identical copy of the calling process: same code, same data, same open files, a separate copy of memory
- `exec()` — replaces the calling process's entire memory image with a new program; same PID, completely different code
- `wait()` — lets a parent pause itself until one of its children finishes, so it can find out how that child terminated

fork(), exec(), and wait() in Practice

```
pid_t pid = fork();

if (pid == 0) {
    /* Child: replace this process's image with /bin/ls */
    execlp("ls", "ls", "-l", NULL);
    perror("exec failed"); /* only reached if exec() failed */
} else if (pid > 0) {
    /* Parent: wait for the child to finish */
    int status;
    wait(&status);
    printf("Child finished.\n");
} else {
    perror("fork failed");
}
```

fork → exec → wait, Traced Through Parent and Child



fork() doesn't run new code — exec() does. If the child never calls exec(), it simply keeps running the exact same code the parent was running, just as an independent process. exec() is the operation that actually swaps in a new program.

Process Termination

A process ends, normally, by calling `exit()` — it finishes its last instruction and asks the OS to reclaim its resources. It can also end abnormally via `abort()`, typically after an unrecoverable error or because its parent (or the OS) decided to kill it.

- Cascading termination — some operating systems terminate every child (and further descendants) when a parent can't continue, rather than leaving them to run unsupervised

Zombie vs. Orphan

Zombie

Has already finished executing and the OS has reclaimed its memory, but its entry still exists in the process table because its parent hasn't yet called `wait()` to collect its exit status.

Orphan

The opposite situation — its parent terminated first while the child was still running. The OS typically reparents it to a designated process (on Unix, usually `init`, PID 1).

Which process is the problem? A zombie is a problem with the child stuck in limbo because the parent hasn't cleaned up after it. An orphan is a problem with the parent being gone while the child is still very much alive — a mirror image of each other, for exactly that reason.

Key Takeaways

- A process is a program in execution — a dynamic entity with its own memory, program counter, and stack — not the same thing as the static program file it was launched from
- A process moves through New, Ready, Running, Waiting, Terminated, driven by admitted, scheduler dispatch, interrupt/time-slice expiry, I/O or event wait, I/O or event completion, and exit
- The PCB is the OS's per-process record of everything needed to pause and later resume that exact process
- Scheduling queues organize processes by state; a context switch moves the CPU between processes and is pure overhead
- `fork()` duplicates a process, `exec()` replaces its program image, and `wait()` lets a parent block until a child finishes
- A zombie has exited but hasn't been reaped by `wait()`; an orphan's parent died first, so it's reparented (typically to `init`, PID 1)